

XRPL

账户基础储备金 (Base Reserve)

功能：每个账户必须持有的最低 XRP 金额，用于支付网络资源占用成本。

基础储备：1 XRP（不可交易，不可用于支付交易费）

所有者储备：每项占用资源（如信任线、代币发行、挂单）需增加 0.2 XRP 储备。

示例：

企业账户持有两个稳定币 2 条信任线，则总储备要求： $1 + 2 \times 0.2 = 1.4\text{XRP}$

Trust Lines

功能：授权的Trust Lines发行方可以启用"Require Auth"标志，将代币所有权限限制在经过批准的账户范围内。

- 防止垃圾代币的强制接收
- 允许实施冻结和授权控制

Trust Lines额度：代表用户最大持有量限制，当用户尝试接收资产时，系统会检查 `Balance + 接收数量` $\leq$ `LimitAmount`。若超出，交易失败。

最终有效额度是双方设置Trust Lines的较小值。

调整额度：使用 `TrustSet` 交易类型，指定发行者地址、资产代码及新额度

IOU精度

并不是通过链上参数直接设置的，而是由发行方和交易双方在操作中隐式定义和管理

- **数值格式**：交易和信任线余额使用字符串表示（如 `"123.456789"`），隐含精度由字符串的小数位数决定。
- **行业惯例**：发行方通常通过外部协议或文档声明代币精度和四舍五入机制。

XRPL 费用模型 vs 以太坊 Gas

以太坊 Gas：

用户需手动指定 `gasLimit` 和 `gasPrice`，通过 `eth_estimateGas` 估算合理值。

XRPL 费用：

费用单位称为 **drops**（1 XRP = 1,000,000 drops）。

费用由网络动态调整，**基础费用**根据网络负载自动计算，用户只需使用节点推荐的当前费用。

无需手动估算复杂参数，交易费用通常由客户端库自动处理。

```
const prepared = await client.autofill(tx); // 自动填充费用、序列号
```

因没有合约执行，因此不需要ethEstimateGas，将prepared发送到kms进行签名后发送节点

```
const result = await client.submitAndWait(signed.tx_blob); // 提交交易，signed是签名后的交易
```

Sequence：账户交易序列号，递增

LastLedgerSequence（交易有效期截止账本序号），若在该指定账本之后仍未确认，交易将永久失效。

Tickets 功能（替代序列机制）：

Tickets 是 XRPL 的一项功能，允许用户为未来的交易**预留独立的序列号**，绕过默认的 `Sequence` 递增规则。它的设计目的是支持**并行交易提交**（无需严格按顺序等待交易确认）。

Tickets 的核心特点

- **独立序列池**：通过 `TicketCreate` 交易生成一批 Ticket（例如生成 10 个），每个 Ticket 有一个唯一的 **Ticket ID**（形如 `3246644` 的整数）。
- **不与主 Sequence 冲突**：使用 Ticket 提交交易时，消耗的是 Ticket ID 而非主 `Sequence`，因此主 `Sequence` 保持不变。
- **灵活性与并发性**：用户可同时提交多个使用不同 Ticket 的交易，无需等待前一笔交易确认。

<https://testnet.xrpl.org/transactions/8CACE0BFF6F49E42F0D9D48FAF6393B016620D7F8CDC326AE90C7F6D0BC7C203>

fee rate：使用 `client.autofill()` 时候会自动获取fee并检查余额

供应量

获取指定 IOU（Issued Currency）的总供应量需要通过查询发行方账户的信任线（Trust Lines）余额来实现

```
const currencyCode = 'USD'; // IOU 货币代码
// 调用 gateway_balances 方法
const response = await client.request({
  command: 'gateway_balances',
  account: ISSUER_ADDRESS,
  ledger_index: 'validated',
  strict: true,
});

// 提取总供应量（obligations）
const totalSupply = response.result.obligations[currencyCode];
console.log(`总发行量: ${totalSupply} ${currencyCode}`);
```

发行方账户初始化流程

1. 发行方通过 AccountSet 交易设置 asfDefaultRipple 标志

作用：支持多跳路径（如通过做市商或网关的中转），支持非直接信任线交易，提高交易效率，若未启用仅支持点对点直接信任线交易。

```
{
  TransactionType: "AccountSet",
  Account: issuer.address, // 发行方地址
  SetFlag: xrp1.AccountSetAsfFlags.asfDefaultRipple // asfDefaultRipple 标志
}
```

2. 发行方通过 AccountSet 交易设置 RequireAuth 标志

作用：启用 RequireAuth 后，其他账户必须 **获得发行方的明确授权**，才能持有该 IOU（即创建信任线）。未经授权的账户无法接收或交易此 IOU。

```
{
  TransactionType: "AccountSet",
  Account: issuer.address, // 发行方地址
  SetFlag: xrp1.AccountSetAsfFlags.asfRequireAuth // asfRequireAuth 标志
}
```

发行方授权用户账户流程

1. 发行方在 **approve** 用户时，需发送 TrustSet 交易授权用户持有和转账自己发行的稳定币（设置 tfSetfAuth 标志）。

注意：针对 USD 和 SGD 需要发送两笔交易，使用 Tickets 机制并发送

```
// 发行方授权 TrustSet
{
  TransactionType: "TrustSet",
  Account: issuer.address, // 发行方地址
  Flags: 65536, // 关键标志
  LimitAmount: {
    currency: CURRENCY_CODE, // 稳定币代号
    issuer: userwallet.address, // 用户地址
    value: "100000000" // 授权的最大额度
  },
  Memos: [
    {
      Memo: {
        MemoType: Buffer.from("PermissionSet").toString("hex").toUpperCase(),
        MemoData: Buffer.from(
          {"issuerAddress": "string", "userAddress": "string", "userPermission": uint256,
          "comments": "string"}
        ).toString("hex").toUpperCase()
      }
    }
  ]
}
```

```

    }
  ]] // 备注,userPermission填9999999
}

```

2. 用户在首次**buy**或**接收稳定币转账**之前，需主动发送 TrustSet 交易，设置指定发行方的信任线，才能接收到相应稳定币

注意：需要讨论在产品位置增加该交易的操作位置

```

// 用户主动发起 TrustSet 请求
{
  TransactionType: "TrustSet",
  Account: userwallet.address, // 用户地址
  LimitAmount: {
    currency: CURRENCY_CODE, // 稳定币代号
    issuer: issuer.address, // 发行方地址
    value: "100000000" // 请求的最大额度
  },
}

```

用户冻结

注意：针对USD和SGD需要发送两笔交易，使用Tickets机制并发送

```

{
  TransactionType: "TrustSet",
  Account: issuer.address,
  LimitAmount: {
    currency: CURRENCY_CODE, // 稳定币代码
    issuer: recipientAddress, // 设置解冻的地址
    value: "0" // 意思是不修改信任线额度
  },
  Flags: 1048576, // 冻结
  Memos: [
    {
      Memo: {
        MemoType: Buffer.from("PermissionSet").toString("hex").toUpperCase(),
        MemoData: Buffer.from(
          {"issuerAddress":"string", "userAddress":"string", "userPermission":uint256,
          "comments":"string"})
          ).toString("hex").toUpperCase()
        }
      }
    ]
  ] // 备注, userPermission填1111111
}

```

用户解冻

注意：针对USD和SGD需要发送两笔交易，使用Tickets机制并发送

```
{
  TransactionType: "TrustSet",
  Account: issuer.address,
  LimitAmount: {
    currency: CURRENCY_CODE, // 稳定币代码
    issuer: recipientAddress, // 设置冻结的地址
    value: "0" // 意思是不修改信任线额度
  },
  Flags: 1048576, // 解冻
  Memos: [
    {
      Memo: {
        MemoType: Buffer.from("PermissionSet").toString("hex").toUpperCase(),
        MemoData: Buffer.from(
          {"issuerAddress":"string", "userAddress":"string", "userPermission":uint256,
          "comments":"string"})
          ).toString("hex").toUpperCase()
        }
      }
    ] // 备注, userPermission填1111111
}
```

用户转账流程

企业用户在发起转账时，发送一笔Payment交易，将指定提现金额的稳定币转入指定接收方账户中

```
{
  TransactionType: "Payment",
  Account: senderWallet.address, // 发送方地址
  Destination: recipientAddress, // 接收方地址
  Amount: {
    currency: CURRENCY_CODE, // 稳定币代号
    value: amount.toString(), // 金额
    issuer: recipientAddress // 发行方地址
  },
  Memos: [
    {
      Memo: {
        MemoType:
Buffer.from("Transfer").toString("hex").toUpperCase(),
        MemoData: Buffer.from(
          {"bid":"string", "from":"string", "to":"string",
          "value":uint256})
          ).toString("hex").toUpperCase()
        }
      }
    ] // 备注
};
```

发行方mint流程

```
{
  TransactionType: "Payment",
  Account: issuerWallet.address, // 发行方地址
  Destination: recipientAddress, // 接收方地址
  Amount: {
    currency: CURRENCY_CODE,
    value: amount.toString(),
    issuer: ISSUER_ADDRESS
  },
  Memos: [
    {
      Memo: {
        MemoType: Buffer.from("Mint").toString("hex").toUpperCase(),
        MemoData: Buffer.from(
          {"bid":"string", "recipient":"string","amount":uint256}
          ).toString("hex").toUpperCase()
      }
    }
  ]
};
```

用户提现流程

1. 企业用户在发起提现请求时，发送一笔Payment交易，将指定提现金额的稳定币转入发行方账户中

```
{
  TransactionType: "Payment",
  Account: senderWallet.address,
  Destination: ISSUER_ADDRESS, // 发行方地址
  Amount: {
    currency: CURRENCY_CODE,
    value: amount.toString(),
    issuer: ISSUER_ADDRESS
  },
  Memos: [
    {
      Memo: {
        MemoType:
Buffer.from("EncashEvent").toString("hex").toUpperCase(),
        MemoData: Buffer.from(
{"bid":"string", "tokenAddress":"string", "encasher":"string", "value":uint256
, "state":"string", "extension":"string"}
          ).toString("hex").toUpperCase()
      }
    }
  ] // 备注， state填"INIT"
```

```
};
```

发行方处理提现请求

1. 发行方确认提现，发送一笔0金额的Payment交易

```
{
  TransactionType: "Payment",
  Account: issuerWallet.address, // 发行方地址
  Destination: recipientAddress, // 提现方地址
  Amount: {
    currency: CURRENCY_CODE,
    value: 0, // 0 金额
    issuer: ISSUER_ADDRESS
  },
  Memos: [
    {
      Memo: {
        MemoType:
Buffer.from("EncashEvent").toString("hex").toUpperCase(),
        MemoData: Buffer.from(
{"bid":"string" ,"tokenAddress":"string ","encasher":"string","value":uint256
,"state":"string ","extension":"string "}
).toString("hex").toUpperCase()
      }
    }
  ] // 备注, state填"ACCEPT"
};
```

2. 发行方拒绝提现，发送一笔Payment交易，退回等额稳定币

```
{
  TransactionType: "Payment",
  Account: issuerWallet.address, // 发行方地址
  Destination: recipientAddress, // 提现方地址
  Amount: {
    currency: CURRENCY_CODE,
    value: amount.toString(),
    issuer: ISSUER_ADDRESS
  },
  Memos: [
    {
      Memo: {
        MemoType:
Buffer.from("EncashEvent").toString("hex").toUpperCase(),
        MemoData: Buffer.from(
{"bid":"string" ,"tokenAddress":"string ","encasher":"string","value":uint256
,"state":"string ","extension":"string "}
).toString("hex").toUpperCase()
      }
    }
  ]
};
```

```
    }  
    ] // 备注, state填"REJECT"  
};
```

链适配器改造

需实现将XRPL赚EVM格式的接口:

ethBlockNumber

```
// Request  
curl -X POST --data '{"jsonrpc":"2.0","method":"eth_blockNumber","params":  
[],"id":83}'  
// Result  
{  
  "id":83,  
  "jsonrpc": "2.0",  
  "result": "0x4b7" // 1207  
}
```

ethGetLogs

```
curl --request POST \  
  --url https://eth-mainnet.g.alchemy.com/v2/docs-demo \  
  --header 'accept: application/json' \  
  --header 'content-type: application/json' \  
  --data '  
{  
  "id": 1,  
  "jsonrpc": "2.0",  
  "method": "eth_getLogs",  
  "params": [  
    {  
      "address": [  
        "0xb59f67a8bff5d8cd03f6ac17265c550ed8f33907"  
      ],  
      "fromBlock": "0x429d3b",  
      "toBlock": "latest",  
    }  
  ]  
}  
,  
  
{  
  "jsonrpc": "2.0",  
  "id": 1,  
  "result": [  
    {  
      "address": "0xb59f67a8bff5d8cd03f6ac17265c550ed8f33907",  
      "blockHash":  
"0x8243343df08b9751f5ca0c5f8c9c0460d8a9b6351066fae0acbd4d3e776de8bb",  
      "blockNumber": "0x429d3b",
```



```

    "data":
    "0x000000000000000000000000000000000000000000000000000000000000000012a05f200",
    "logIndex": "0x56",
    "removed": false,
    "topics": [
      "0xdddf252ad1be2c89b69c2b068fc378daa952ba7f163c4a11628f55a4df523b3ef",
      "0x0000000000000000000000000000000000000000000000000000000000000000b46c2526e227482e2ebb8f4c69e4674d262e75",
      "0x000000000000000000000000000000000000000000000000000000000000000054a2d42a40f51259dedd1978f6c118a0f0eff078"
    ],
    "transactionHash":
    "0xab059a62e22e230fe0f56d8555340a29b2e9532360368f810595453f6fdd213b",
    "transactionIndex": "0xac"
  }
]
}

```

ethGetTransactionByHash

```

// Request
curl -X POST --data
'{"jsonrpc":"2.0","method":"eth_getTransactionByHash","params":
["0x88df016429689c079f3b2f6ad39fa052532c56795b733da78a91ebe6a713944b"],"id":1}'
// Result
{
  "jsonrpc":"2.0",
  "id":1,
  "result":{

    "blockHash":"0x1d59ff54b1eb26b013ce3cb5fc9dab3705b415a67127a003c3e61eb445bb8df2",
    "blockNumber":"0x5daf3b", // 6139707
    "from":"0xa7d9ddbe1f17865597fbd27ec712455208b6b76d",
    "gas":"0xc350", // 50000
    "gasPrice":"0x4a817c800", // 20000000000
    "hash":"0x88df016429689c079f3b2f6ad39fa052532c56795b733da78a91ebe6a713944b",
    "input":"0x68656c6c6f21",
    "nonce":"0x15", // 21
    "to":"0xf02c1c8e6114b1dbe8937a39260b5b0a374432bb",
    "transactionIndex":"0x41", // 65
    "value":"0xf3dbb76162000", // 4290000000000000
    "v":"0x25", // 37
    "r":"0x1b5e176d927f8e9ab405058b2d2457392da3e20f328b16ddabcebc33eaac5fea",
    "s":"0x4ba69724e8f69de52f0125ad8b3c5c2cef33019bac3249e2c0a2192766d1721c"
  }
}

```

ethGetTransactionReceipt

```

// Request
curl -X POST --data
'{"jsonrpc":"2.0","method":"eth_getTransactionReceipt","params":
["0x85d995eba9763907fdf35cd2034144dd9d53ce32cbec21349d4b12823c6860c5"],"id":1}'
// Result
{

```



```

    "number": "0x1b4",
    "parentHash":
"0xe99e022112df268087ea7eafaf4790497fd21dbeeb6bd7a1721df161a6657a54",
    "receiptsRoot":
"0x56e81f171bcc55a6ff8345e692c0f86e5b48e01b996cad001622fb5e363b421",
    "sha3Uncles":
"0x1dcc4de8dec75d7aab85b567b6ccd41ad312451b948a7413f0a142fd40d49347",
    "size": "0x220",
    "stateRoot":
"0xddc8b0234c2e0cad087c8b389aa7ef01f7d79b2570bccb77ce48648aa61c904d",
    "timestamp": "0x55ba467c",
    "totalDifficulty": "0x78ed983323d",
    "transactions": [
    ],
    "transactionsRoot":
"0x56e81f171bcc55a6ff8345e692c0f86e5b48e01b996cad001622fb5e363b421",
    "uncles": [
    ]
}
}

```

getTransaction (queryTxStatus)

KMS改造

增加支持xrpl网络的签名、多签流程相关接口，对接xrpl的sdk签名，接口格式待定

go sdk

<https://github.com/Peersyst/xrpl-go>

TopUP

后端转账xrp的开发。

jave sdk

<https://github.com/XRPLF/xrpl4j>